

--	--	--	--	--	--	--	--	--	--

RV COLLEGE OF ENGINEERING®
(An Autonomous Institution Affiliated to VTU)
VI Semester B. E. Examinations Sept/Oct – 2024
Artificial Intelligence and Machine Learning

ARTIFICIAL NEURAL NETWORKS AND DEEP LEARNING

Time: 03 Hours

Maximum Marks: 100

Instructions to candidates:

- Answer all questions from Part A. Part A questions should be answered in first three pages of the answer book only.
- Answer FIVE full questions from Part B. In Part B question number 2 is compulsory. Answer any one full question from 3 and 4, 5 and 6, 7 and 8, 9 and 10.

PART-A

M BT CO

1	1.1	Define Hebbian learning and articulate the rule it follows.	02	2	1										
	1.2	What is the concept of adaption in the context of neural networks?	02	1	1										
	1.3	Given an input vector $x[1,2]$, $f\{x\}= [1,2]$ and a desired output $d=3$, assuming the initial weights of a single – layer perceptron are $w=[0.5, -0.5]$. Compute the output of the perceptron using a linear activation function.	02	2	1										
	1.4	In a batch mode, if the total error after one epoch is $E=5$ for 10 samples and in sequential mode the error after the first 5 samples is $E_5=3E$, what is the average error per sample in each mode?	02	1	1										
	1.5	i. In a convolution layer _____. refers to the addition of pixels around the border of an image to control the spatial dimensions of the output. ii. The process of _____ in a CNN reduce the spatial dimensions of the input volume and helps to make the representations more invariant to small translations	02	2	1										
	1.6	Complete the answer: Local Response Normalization (LRN) is a technique used in CNNs to _____.	02	1	1										
	1.7	Match each concept or application related to Recurrent Neural Networks (RNNs) in Column A with its corresponding description or example in Column B.													
		<table><tr><th>Column A</th><th>Column B</th></tr><tr><td>I. Expressiveness of Recurrent Networks</td><td>i. Similar to LSTM but with a simplified structure for faster computation and fewer parameters.</td></tr><tr><td>II. Long Short-Term Memory (LSTM)</td><td>ii. Application involving generating textual descriptions for images.</td></tr><tr><td>III. Application to Automatic image Captioning</td><td>iii. A model’s ability to represent complex dependencies and relationships within sequences.</td></tr><tr><td>IV. Gated Recurrent Units (GRUs)</td><td>iv. RNN variant that addresses the vanishing gradient problem using gates for better long-term memory retention</td></tr></table>	Column A	Column B	I. Expressiveness of Recurrent Networks	i. Similar to LSTM but with a simplified structure for faster computation and fewer parameters.	II. Long Short-Term Memory (LSTM)	ii. Application involving generating textual descriptions for images.	III. Application to Automatic image Captioning	iii. A model’s ability to represent complex dependencies and relationships within sequences.	IV. Gated Recurrent Units (GRUs)	iv. RNN variant that addresses the vanishing gradient problem using gates for better long-term memory retention	02	1	1
Column A	Column B														
I. Expressiveness of Recurrent Networks	i. Similar to LSTM but with a simplified structure for faster computation and fewer parameters.														
II. Long Short-Term Memory (LSTM)	ii. Application involving generating textual descriptions for images.														
III. Application to Automatic image Captioning	iii. A model’s ability to represent complex dependencies and relationships within sequences.														
IV. Gated Recurrent Units (GRUs)	iv. RNN variant that addresses the vanishing gradient problem using gates for better long-term memory retention														

1.8	State whether the below sentences are true or false i. Echo-State Networks are a type of RNN that relies on a large, fixed reservoir of neurons with random weights and only trains the output layer. ii. RNN are ineffective for handwriting recognition tasks due to their inability to learn temporal patterns in sequential data.	02	4	1
1.9	In the context of Multi-Armed Bandits, what is the Greedy Algorithm? Give one limitation.	02	2	2
1.10	In the context of self-driving cars, how does reinforcement learning contribute to decision making?	02	1	1

PART-B

2	a	Articulate the working of multi-layer feed forward and RNN architectures in various applications through necessary diagrams.	08	2	2																																																	
	b	Analyze the key differences between supervised and unsupervised and reinforcement machine learning.	08	2	1																																																	
3	a	What is unconstrained optimization problem? Explain the working of Least Mean Square method.	08	2	3																																																	
	b	With a neat sketch elaborate the working of multi-layer perceptron.	08	2	2																																																	
		OR																																																				
4	a	For back propagation algorithm, analyze the different ways of defining stopping criteria.	08	2	3																																																	
	b	With relevant example discuss how back propagation can be a solution for XOR problem.	08	2	2																																																	
5	a	AlexNet is implemented by researchers for automation of recognition of casting surface defects and the accuracy is 86%. Analyze the given case study and list out the reasons for better performance with respect to the case study.	08	2	2																																																	
	b	For the given input perform different convolution operations considering Zew padding and stride=1 <table border="1"><tr><td>6</td><td>3</td><td>4</td><td>4</td><td>5</td><td>0</td><td>3</td></tr><tr><td>4</td><td>7</td><td>4</td><td>0</td><td>4</td><td>0</td><td>4</td></tr><tr><td>7</td><td>0</td><td>2</td><td>3</td><td>4</td><td>5</td><td>2</td></tr><tr><td>3</td><td>7</td><td>5</td><td>0</td><td>3</td><td>0</td><td>7</td></tr><tr><td>5</td><td>8</td><td>1</td><td>2</td><td>5</td><td>4</td><td>2</td></tr><tr><td>8</td><td>0</td><td>1</td><td>0</td><td>6</td><td>0</td><td>0</td></tr><tr><td>6</td><td>4</td><td>1</td><td>3</td><td>0</td><td>4</td><td>5</td></tr></table>	6	3	4	4	5	0	3	4	7	4	0	4	0	4	7	0	2	3	4	5	2	3	7	5	0	3	0	7	5	8	1	2	5	4	2	8	0	1	0	6	0	0	6	4	1	3	0	4	5	08	3	2
6	3	4	4	5	0	3																																																
4	7	4	0	4	0	4																																																
7	0	2	3	4	5	2																																																
3	7	5	0	3	0	7																																																
5	8	1	2	5	4	2																																																
8	0	1	0	6	0	0																																																
6	4	1	3	0	4	5																																																
		OR																																																				
6	a	Assume a series of data stored as PET scan images with reference to classifying the image as presence or absence of cancer causing tissues. The number of images is less for training. Identify and justify the technique used to increase the training set and explain the process in detail.	08	2	2																																																	

b	Elaborate how pooling can be used to detect the edges in an image. For the given input matrix below <i>Input 6 x 6 matrix</i> <table><tr><td>3</td><td>0</td><td>1</td><td>2</td><td>7</td><td>4</td></tr><tr><td>1</td><td>5</td><td>8</td><td>9</td><td>3</td><td>1</td></tr><tr><td>2</td><td>7</td><td>2</td><td>5</td><td>1</td><td>3</td></tr><tr><td>0</td><td>1</td><td>3</td><td>1</td><td>7</td><td>8</td></tr><tr><td>4</td><td>2</td><td>1</td><td>6</td><td>2</td><td>8</td></tr><tr><td>2</td><td>4</td><td>5</td><td>2</td><td>3</td><td>9</td></tr></table> <i>Filter 3 X3</i> <table><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr></table> Apply max and average pooling considering the size of 3 × 3.	3	0	1	2	7	4	1	5	8	9	3	1	2	7	2	5	1	3	0	1	3	1	7	8	4	2	1	6	2	8	2	4	5	2	3	9	1	0	-1	1	0	-1	1	0	-1	08	3	2
3	0	1	2	7	4																																												
1	5	8	9	3	1																																												
2	7	2	5	1	3																																												
0	1	3	1	7	8																																												
4	2	1	6	2	8																																												
2	4	5	2	3	9																																												
1	0	-1																																															
1	0	-1																																															
1	0	-1																																															
7	a	Explain the concept of Back Propagation Through Time (BPTT) and its role in training RNNs. How does it address the issue of learning long term dependence?	06	3	3																																												
	b	Articulate the role of RNNs in natural language processing (NLP) tasks. Identify and elaborate an appropriate RNN model for language translation with an example.	10	2	2																																												
		OR																																															
8	a	Analyze the differences between CNN and RNN	06	3	3																																												
	b	With suitable example and necessary diagrams illustrate the working of LSTM	10	2	2																																												
9	a	Demonstrate the basic framework of Reinforcement learning (RL) and key components involved in the learning process. How do challenges such as exploration versus exploitation impact RL algorithms?	06	2	2																																												
	b	With an example, illustrate how a neural network can be used as a function approximator in an RL setting, such as playing Atari games. Describe how the neural network estimates the value function and benefits of this approach.	10	3	3																																												
		OR																																															
10	a	Consider the example “The player chases the ball”. Apply Markov-Decision process and elaborate the steps in detail.	06	2	2																																												
	b	Analyze how DL techniques are used in building conversational systems such as chatbots. Also list the benefits and limitations of using DL for natural language understanding and generation.	10	2	3																																												

Solutions: Artificial Neural Networks and Deep Learning

RV COLLEGE OF ENGINEERING® VI Semester B. E. Examinations Sept/Oct – 2024 Quiz: PART-A

1.1 Define Hebbian learning and articulate the rule it follows.

Definition: Hebbian learning is a neuroscientific theory proposing an explanation for the adaptation of neurons in the brain during the learning process. Often summarized by the phrase "**Cells that fire together, wire together,**" it postulates that the synaptic connection between two neurons strengthens if the neurons are active simultaneously.

The Hebbian Rule: Mathematically, the change in the weight (w_{ij}) connecting a pre-synaptic neuron j to a post-synaptic neuron i is proportional to the product of their activation values.

$$\Delta w_{ij} = \eta \cdot y_i \cdot x_j$$

Where:

- Δw_{ij} is the change in weight.
- η (eta) is the learning rate.
- y_i is the output of the post-synaptic neuron.
- x_j is the input from the pre-synaptic neuron.

1.2 What is the concept of adaptation in the context of neural networks?

Adaptation refers to the ability of a neural network to modify its internal parameters—specifically the **synaptic weights** and **biases**—in response to changes in its environment or input data.

In a learning context, adaptation is the process by which the network minimizes a cost function (error) by iteratively adjusting these parameters. This allows the network to "learn" the mapping between inputs and desired outputs, improving its performance over time (epochs).

1.3 Perceptron Output Computation

Given:

- Input vector: $x = [1, 2]^T$
- Initial weights: $w = [0.5, -0.5]^T$
- Desired output: $d = 3$
- Activation Function: Linear

Calculation: The induced local field (weighted sum), denoted as v , is calculated as the dot product of the weight vector and the input vector:

$$v = \sum_{i=1}^n w_i x_i$$

$$v = (0.5 \times 1) + (-0.5 \times 2)$$

$$v = 0.5 - 1.0$$

$$v = -0.5$$

Since the activation function is **Linear** ($\phi(v) = v$):

$$\text{Output } y = -0.5$$

1.4 Batch Mode vs. Sequential Mode Error Calculation

Given:

1. **Batch Mode:** Total Error (E_{total}) = 5 for 10 samples.
2. **Sequential Mode:** Error after the first 5 samples (E_5) = 3 (Assuming "3E" in the prompt text is a typo for the value 3.0, or implies an accumulated error of 3).

Formula:

$$\text{Average Error} = \frac{\text{Total Accumulated Error}}{\text{Number of Samples}}$$

Calculations:

- **Batch Mode:**

$$E_{avg} = \frac{5}{10} = 0.5$$

- **Sequential Mode (for the first 5 samples):**

$$E_{avg} = \frac{3}{5} = 0.6$$

1.5 Convolutional Neural Networks (CNN) Concepts

i. Padding In a convolution layer, **Padding** refers to the addition of pixels around the border of an image to control the spatial dimensions of the output. (Specifically, *Zero-padding* is used to preserve the spatial size after convolution).

ii. Pooling The process of **Pooling** (or Subsampling) in a CNN reduces the spatial dimensions of the input volume and helps to make the representations more invariant to small translations.

1.6 Local Response Normalization (LRN)

Complete the answer: Local Response Normalization (LRN) is a technique used in CNNs to perform lateral inhibition.

Detail: It encourages competition between neurons at the same spatial location but in different feature maps (channels). By dampening the responses of neurons that are uniformly large and amplifying those that are essentially peaks relative to their neighbors, it helps the network generalize better and highlights high-frequency features.

1.7 RNN Concepts Matching

Column A	Column B	Correct Match
I. Expressiveness of Recurrent Networks	iii. A model's ability to represent complex dependencies and relationships within sequences.	I - iii
II. Long Short-Term Memory (LSTM)	iv. RNN variant that addresses the vanishing gradient problem using gates for better long-term memory retention.	II - iv
III. Application to Automatic Image Captioning	ii. Application involving generating textual descriptions for images.	III - ii
IV. Gated Recurrent Units (GRUs)	i. Similar to LSTM but with a simplified structure for faster computation and fewer parameters.	IV - i

1.8 True or False

i. Echo-State Networks are a type of RNN that relies on a large, fixed reservoir of neurons with random weights and only trains the output layer.

- **Status: TRUE**
- *Explanation:* In ESNs (Reservoir Computing), the internal weights are fixed and random; only the readout (output) weights are trained, usually via linear regression.

ii. RNN are ineffective for handwriting recognition tasks due to their inability to learn temporal patterns in sequential data.

- **Status: FALSE**
- *Explanation:* RNNs, especially LSTMs and GRUs, are highly effective for handwriting recognition because handwriting is fundamentally a sequence of strokes over time (temporal data).

1.9 Multi-Armed Bandits: Greedy Algorithm

Definition: In the context of the Multi-Armed Bandit problem, the **Greedy Algorithm** is a strategy where the agent always selects the action (arm) with the highest estimated value (Q -value) based on current knowledge. It exploits the current best option 100% of the time.

$$A_t = \arg \max_a Q_t(a)$$

Limitation: Lack of Exploration: The Greedy Algorithm creates a pure exploitation approach. If the initial estimates are incorrect or if a suboptimal arm gets lucky early on, the algorithm may lock onto a suboptimal action and never discover that another arm actually yields a higher reward. It fails to explore the environment to find the global optimum.

1.10 Reinforcement Learning in Self-Driving Cars

Contribution to Decision Making: In self-driving cars, Reinforcement Learning (RL) contributes to decision-making by enabling the vehicle (agent) to learn optimal control policies through trial and error interactions with the environment.

1. **Reward System:** The car receives rewards for desirable behaviors (staying in lane, maintaining speed, smooth braking) and penalties for undesirable ones (collisions, off-roading, hard stops).
2. **Complex Maneuvers:** Unlike rule-based systems, RL is particularly good at handling high-dimensional, complex scenarios such as:
 - **Overtaking decisions:** Deciding when it is safe to pass another car.
 - **Merging:** Navigating into highway traffic.
 - **Intersection negotiation:** Handling 4-way stops or roundabouts.
3. **Optimization:** The RL agent learns a policy $\pi(s)$ that maps current sensor states (camera, LiDAR) to actions (steering, throttle) to maximize the cumulative long-term reward, ensuring safety and efficiency.

Solutions: Artificial Neural Networks and Deep Learning

RV COLLEGE OF ENGINEERING® VI Semester B. E. Examinations Sept/Oct – 2024 Quiz:
PART-B

2a. Articulate the working of multi-layer feed forward and RNN architectures in various applications through necessary diagrams. (08 Marks)

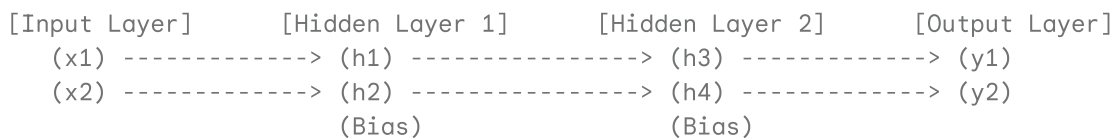
This answer is divided into two parts: Multi-Layer Feed Forward Networks (MLFFN) and Recurrent Neural Networks (RNN).

1. Multi-Layer Feed Forward Networks (MLFFN)

Working Principle: A Multi-Layer Feed Forward Network consists of an input layer, one or more hidden layers, and an output layer. The term "Feed Forward" implies that information flows in only one direction: forward, from the input nodes, through the hidden nodes, and to the output nodes. There are no cycles or loops in the network.

- **Structure:**
 - **Input Layer:** Receives the raw data features. No computation happens here.
 - **Hidden Layers:** Perform the core computations. Each neuron calculates a weighted sum of inputs plus a bias, passes it through a non-linear activation function (like ReLU or Sigmoid), and sends it to the next layer. This allows the network to learn complex, non-linear boundaries.
 - **Output Layer:** Produces the final prediction (e.g., probability distribution for classification).
- **Training (Backpropagation):** The network learns by calculating the error at the output and propagating it backward to update weights using gradient descent.

Conceptual Diagram (MLFFN):



(Information flows strictly left to right)

Applications:

1. **Image Classification (Basic):** Classifying static images (e.g., MNIST digit recognition) where spatial hierarchy is less critical than simple pixel combinations.
2. **Tabular Data Analysis:** Predicting housing prices or customer churn based on fixed sets of features.

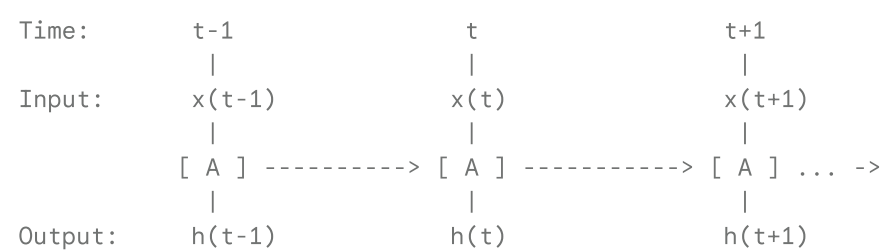
- 3. **Function Approximation:** Approximating complex mathematical functions in engineering controls.

2. Recurrent Neural Networks (RNN)

Working Principle: RNNs are designed for sequential data. Unlike MLFFNs, RNNs have connections that form directed cycles (loops). This allows the output from the previous step to be fed as input to the current step.

- **Memory (Hidden State):** The key feature is the "Hidden State" (h_t), which acts as memory. It captures information about what has been calculated so far.
- **Temporal Processing:** At each time step t , the neuron receives the current input (x_t) and the previous hidden state (h_{t-1}).
- **Equation:** $h_t = f(W \cdot x_t + U \cdot h_{t-1} + b)$

Conceptual Diagram (Unrolled RNN):



(The block 'A' represents the neural network layer looking at itself from the previous step)

Applications:

- 1. **Natural Language Processing (NLP):**
 - *Language Modeling:* Predicting the next word in a sentence (e.g., Autocomplete).
 - *Machine Translation:* Translating a sentence from English to French (Sequence-to-Sequence).
- 2. **Time Series Prediction:** Stock market forecasting or weather prediction where past values influence future values.
- 3. **Speech Recognition:** Converting audio waves (sequence of signals) into text.

2b. Analyze the key differences between supervised, unsupervised, and reinforcement machine learning. (08 Marks)

These three paradigms represent the fundamental ways machines learn from data.

Feature	Supervised Learning	Unsupervised Learning	Reinforcement Learning (RL)
---------	---------------------	-----------------------	-----------------------------

Definition	The model learns from a labeled dataset, mapping inputs to known targets.	The model learns from unlabeled data, finding hidden structures or patterns.	The agent learns by interacting with an environment to maximize a reward signal.
Input Data	Labeled Data: Pairs of <i>(Input, CorrectOutput)</i> .	Unlabeled Data: Only Input data is provided; no target tags.	States & Rewards: No predefined data; data is generated via interaction.
Feedback Mechanism	Direct: The model is explicitly told the correct answer (error signal) during training.	None: There is no "correct" answer provided; the model evaluates internal consistency.	Delayed: The agent receives a scalar "reward" or "penalty" after actions, often steps later.
Goal	To predict the output for new, unseen inputs accurately.	To explore the data and find structure, clusters, or simplified representations.	To learn a policy (sequence of actions) that maximizes cumulative reward over time.
Common Algorithms	Linear Regression, Logistic Regression, SVM, Neural Networks (Backprop).	K-Means Clustering, PCA (Principal Component Analysis), Autoencoders.	Q-Learning, Deep Q-Networks (DQN), Policy Gradients, Multi-Armed Bandit.
Example Application	- Spam detection in email. - Diagnosing disease from X-rays (Classification).	- Customer segmentation in marketing. - Anomaly detection in credit card fraud.	- Playing Chess or Go (AlphaGo). - Robot navigation. - Self-driving car control.
Core Challenge	Overfitting (memorizing data) and requiring expensive labeled datasets.	Interpreting the results (clusters) and lack of objective accuracy metrics.	Exploration vs. Exploitation trade-off and credit assignment (delayed rewards).

3a. What is unconstrained optimization problem? Explain the working of Least Mean Square method. (08 Marks)

Unconstrained Optimization Problem: An unconstrained optimization problem involves finding the local or global minimizers (or maximizers) of an objective function $f(x)$ where the decision variables x can take any value from the domain (usually $\mathbb{R}^n$) without any restrictions, boundaries, or constraints (such as $g(x) \leq 0$ or $h(x) = 0$).

Mathematically:

Minimize $f(\mathbf{x})$ where $\mathbf{x} \in \mathbb{R}^n$

In neural networks, training is often framed as an unconstrained optimization problem where we seek to minimize the Loss Function $J(w)$ with respect to the weights w .

Least Mean Square (LMS) Method: The LMS method, also known as the **Widrow-Hoff rule** or **Delta rule**, is an adaptive algorithm used to adjust the weights of a linear neuron (ADALINE) to minimize the error between the actual output and the desired output. It is a stochastic gradient descent approach.

- **Objective:** Minimize the instantaneous squared error cost function:

$$E(n) = \frac{1}{2}e^2(n) = \frac{1}{2}(d(n) - y(n))^2$$

Where $d(n)$ is the desired response and $y(n) = \mathbf{w}^T(n)\mathbf{x}(n)$ is the actual output at step n .

- **Working Principle (Algorithm):**

1. **Initialization:** Set weights $\mathbf{w}(0)$ to small random values.
2. **For each iteration n :**
 - **Forward Pass:** Compute the output $y(n)$ using current weights.

$$y(n) = \sum w_i(n)x_i(n)$$

- **Error Computation:** Calculate the error signal.

$$e(n) = d(n) - y(n)$$

- **Weight Update:** Adjust the weights in the direction opposite to the gradient of the squared error.

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \eta \cdot e(n) \cdot \mathbf{x}(n)$$

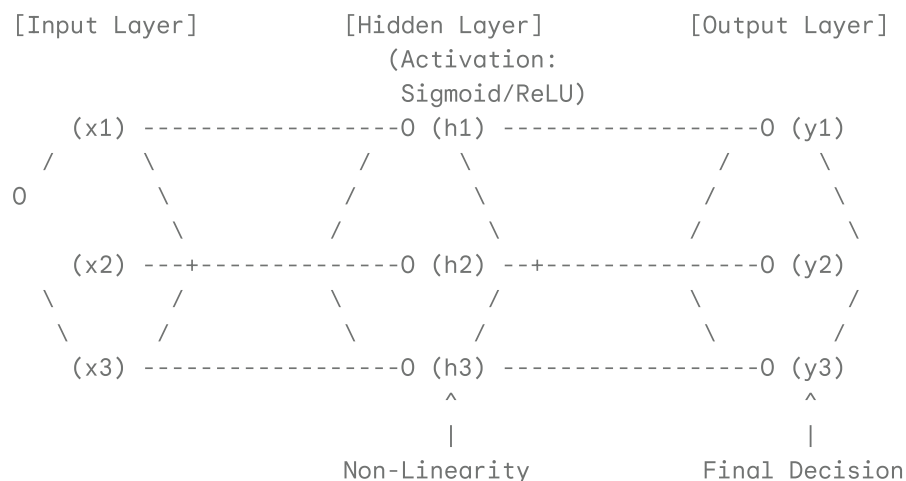
Where η is the learning rate.

- **Key Characteristic:** Unlike standard Gradient Descent which averages the error over the entire dataset, LMS updates weights after *each* sample. This makes it computationally efficient and capable of tracking changes in non-stationary environments.

3b. With a neat sketch elaborate the working of multi-layer perceptron. (08 Marks)

Multi-Layer Perceptron (MLP): An MLP is a class of feedforward artificial neural networks. It consists of at least three layers of nodes: an input layer, a hidden layer, and an output layer. Unlike the single-layer perceptron, MLPs can distinguish data that is not linearly separable.

Sketch:



Working:

1. **Input Propagation:** Input features $(x_1, x_2, \dots)$ are fed into the input layer.
2. **Weighted Sum & Bias:** Each neuron in the hidden layer calculates the weighted sum of inputs plus a bias term.

$$v_j = \sum w_{ji}x_i + b_j$$

3. **Non-Linear Activation:** The key to the MLP's power is the activation function (like Sigmoid, Tanh, or ReLU) applied in the hidden layers. This transforms the weighted sum (v_j) into an output $(y_j = \phi(v_j))$. This non-linearity allows the network to learn complex patterns and boundaries.
4. **Output Generation:** The signals propagate to the output layer, which aggregates them to produce the final prediction.
5. **Learning (Backpropagation):** The network is trained using Backpropagation, where the error at the output is calculated, and gradients are passed backward through the layers to update weights, minimizing the total error.

4a. For back propagation algorithm, analyze the different ways of defining stopping criteria. (08 Marks)

The Backpropagation algorithm uses an iterative process (Gradient Descent) to minimize error. Deciding when to stop training is crucial to prevent overfitting and ensure efficiency.

Stopping Criteria:

1. **Total Number of Epochs:**
 - *Method:* Stop after a fixed, pre-defined number of iterations (e.g., 10,000 epochs).
 - *Analysis:* Simple to implement but rigid. If set too low, the model underfits; if too high, it wastes time and may overfit.
2. **Error Threshold (Cost Function Value):**

- *Method:* Stop when the training error (Mean Squared Error) falls below a small value ϵ (e.g., 0.001).
- *Analysis:* Ensures the model has learned the training data well. However, for noisy data, the error might never reach this threshold, causing an infinite loop.

3. Gradient Magnitude:

- *Method:* Stop when the norm of the gradient vector is close to zero ($\|\nabla J(w)\| < \epsilon$).
- *Analysis:* Indicates that the algorithm has reached a local minimum or a plateau (saddle point).

4. Relative Change in Error:

- *Method:* Stop if the change in error between consecutive epochs is negligible.
- *Analysis:* Useful when the learning curve flattens out, indicating diminishing returns for further training.

5. Early Stopping (Generalization Loss):

- *Method:* Monitor the error on a separate **Validation Set** (not used for training).
- *Analysis:* Ideally, training error decreases continuously. However, validation error decreases initially but eventually starts increasing when the model begins to overfit (memorize noise). Training is stopped at the point where validation error is minimum. This is the most robust criterion for generalization.

4b. With relevant example discuss how back propagation can be a solution for XOR problem. (08 Marks)

The XOR Problem: The XOR (Exclusive OR) function is a classic example of a problem that is **not linearly separable**.

- Inputs: (0,0), (0,1), (1,0), (1,1)
- Outputs: 0, 1, 1, 0 A single straight line cannot separate the outputs 1 from the outputs 0. Therefore, a single-layer perceptron cannot solve it.

Backpropagation Solution (MLP Architecture): To solve XOR, we need a Multi-Layer Perceptron (MLP) with at least one hidden layer. A common architecture is **2-2-1**:

- 2 Input Neurons (x_1, x_2)
- 2 Hidden Neurons (h_1, h_2)
- 1 Output Neuron (y)

How it works:

1. **Transformation of Space:** The hidden neurons map the original input space into a new feature space.
 - Neuron h_1 can learn to activate when *at least one* input is 1 (OR-like behavior).
 - Neuron h_2 can learn to activate when *both* inputs are 1 (AND-like behavior).

2. **Non-Linearity:** Using a non-linear activation function (e.g., Sigmoid) is mandatory. Without it, the multi-layer network collapses into a single linear layer.
3. **Decision Boundary:** The output neuron combines the signals from h_1 and h_2 . It effectively computes something like $h_1 - h_2$.
 - If (0,0) -> h_1 low, h_2 low -> Output Low (0)
 - If (0,1) or (1,0) -> h_1 high, h_2 low -> Output High (1)
 - If (1,1) -> h_1 high, h_2 high (inhibitory connection) -> Output Low (0)

Solutions: Artificial Neural Networks and Deep Learning

RV COLLEGE OF ENGINEERING® VI Semester B. E. Examinations Sept/Oct – 2024 Quiz:
PART-C

5a. AlexNet is implemented by researchers for automation of recognition of casting surface defects and the accuracy is 86%. Analyze the given case study and list out the reasons for better performance with respect to the case study. (08 Marks)

Analysis of the Case Study: In the context of industrial automation, detecting defects on casting surfaces (like blowholes, shrinkage, or cracks) is a visual task traditionally performed by humans. Implementing AlexNet, a deep Convolutional Neural Network (CNN), automates this process. An accuracy of 86% indicates the model successfully captures the complex, non-linear texture patterns associated with defects.

Reasons for Better Performance (AlexNet Features):

- 1. Deep Architecture (Depth):** Unlike earlier shallow networks, AlexNet utilizes 8 layers (5 convolutional, 3 fully connected). This depth allows the network to learn a hierarchy of features:
 - *Lower layers* detect simple edges and surface irregularities (critical for cracks).
 - *Higher layers* detect complex textures and shapes (critical for blowholes or slag inclusions).
- 2. ReLU Activation (Rectified Linear Unit):** AlexNet replaced the Sigmoid/Tanh functions with ReLU ($f(x) = \max(0, x)$).
 - *Benefit:* It solves the vanishing gradient problem, allowing the network to train faster and converge better, which is crucial when learning subtle defect features from high-resolution surface images.
- 3. Data Augmentation:** Casting defect datasets are often small. AlexNet employs augmentation techniques (cropping, flipping, color jittering) during training.
 - *Benefit:* This artificially expands the dataset, making the model invariant to the orientation or lighting conditions of the casting surface, reducing overfitting.
- 4. Dropout:** Implemented in the fully connected layers (typically 0.5 probability), Dropout randomly "turns off" neurons during training.
 - *Benefit:* This prevents complex co-adaptations where one neuron relies too much on another. It forces the network to learn robust features effectively, ensuring that the detection of a defect relies on the visual evidence of the defect itself, not noise.
- 5. Overlapping Pooling:** AlexNet uses max pooling with a stride smaller than the window size (e.g., 3x3 window with stride 2).
 - *Benefit:* This reduces the size of the representation while minimizing information loss, retaining the strongest textures (defects) while providing slight translational

invariance.

5b. For the given input perform different convolution operations considering Zero padding and stride=1. (08 Marks)

Given:

- **Input Image (I):** 7x7 Matrix
- **Padding (P):** Zero padding (assumed $P = 1$ for 3x3 kernel to maintain size, or just standard zero padding).
- **Stride (S):** 1

Input Matrix:

$$\begin{bmatrix} 6 & 3 & 4 & 4 & 5 & 0 & 3 \\ 4 & 7 & 4 & 0 & 4 & 0 & 4 \\ 7 & 0 & 2 & 3 & 4 & 5 & 2 \\ 3 & 7 & 5 & 0 & 3 & 0 & 7 \\ 5 & 8 & 1 & 2 & 5 & 4 & 2 \\ 8 & 0 & 1 & 0 & 6 & 0 & 0 \\ 6 & 4 & 1 & 3 & 0 & 4 & 5 \end{bmatrix}$$

*Note: The problem asks for "different convolution operations" but does not provide specific filters. We will demonstrate the operation using a standard **Edge Detection Filter**.*

Assumed Filter (K) - Vertical Edge Detector:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Process:

1. **Padding:** Add a border of zeros around the 7x7 matrix. The new size is 9x9.
2. **Convolution:** Slide the 3x3 filter over the padded image with stride 1.
 - Formula for position (i, j) : $\sum \sum (I_{slice} \times K)$

Sample Calculation (Top-Left Corner - First valid position): We place the center of the kernel on the first pixel (6) of the original image (which corresponds to row 1, col 1 in padded version).

Top-left 3x3 patch from padded input (centered at 6):

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 6 & 3 \\ 0 & 4 & 7 \end{bmatrix}$$

Wait: With zero padding $P = 1$, the first calculation is usually centered on the top-left pixel. Let's compute the value for the first output pixel $(0, 0)$.

Overlay Filter on Padded Input (Top-Left):

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 6 & 3 \\ 0 & 4 & 7 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Calculation: $(0 \times 1) + (0 \times 0) + (0 \times -1) + (0 \times 1) + (6 \times 0) + (3 \times -1) + (0 \times 1) + (4 \times 0) + (7 \times -1)$

$= 0 + 0 + 0 + 0 + 0 - 3 + 0 + 0 - 7$ **Output(0,0) = -10**

Sample Calculation (Middle of matrix - e.g., row 2, col 2 of original): Original patch centered at 7 (row 1, col 1):

$$\begin{bmatrix} 6 & 3 & 4 \\ 4 & 7 & 4 \\ 7 & 0 & 2 \end{bmatrix}$$

Calculation: $(6 \times 1) + (3 \times 0) + (4 \times -1) + (4 \times 1) + (7 \times 0) + (4 \times -1) + (7 \times 1) + (0 \times 0) + (2 \times -1)$

$= (6 - 4) + (4 - 4) + (7 - 2) = 2 + 0 + 5$ **Output = 7**

This process is repeated for every pixel. The output matrix will be 7x7 (since $P = 1$ and $S = 1$ preserves dimensions for a 3x3 kernel).

6a. Assume a series of data stored as PET scan images with reference to classifying the image as presence or absence of cancer causing tissues. The number of images is less for training. Identify and justify the technique used to increase the training set and explain the process in detail. (08 Marks)

Technique Identified: Data Augmentation

Justification: In medical imaging (like PET scans), collecting labeled data is expensive, time-consuming, and limited by patient privacy. A small dataset typically leads to **overfitting**, where the model memorizes specific training examples but fails to generalize to new patients. Data Augmentation artificially creates new training samples from the existing ones by applying transformations that do not change the underlying label (presence/absence of cancer), thereby increasing dataset diversity and model robustness.

Process in Detail:

1. Geometric Transformations:

- **Rotation:** Rotating the PET scan image by small angles (e.g., $\pm 5^\circ$, 15°). Since tumors can appear at any orientation, this teaches the model rotational invariance.
- **Flipping/Mirroring:** Horizontally or vertically flipping the image. A tumor on the left is structurally similar to one on the right.

- **Translation (Shifting):** Shifting the image along the X or Y axis. This helps the network learn that the position of the tumor in the frame does not define the cancer itself.

2. Intensity Transformations:

- **Contrast/Brightness Adjustment:** Varying the intensity levels simulates different scanner calibrations or lighting conditions.
- **Noise Injection:** Adding small amounts of Gaussian noise helps the model ignore high-frequency artifacts and focus on the tissue structure.

3. Elastic Deformation (Advanced):

- Applying random grid distortions to simulate the natural variations in organ shape and tissue elasticity, which is highly relevant in medical imaging.

By applying these techniques, a dataset of 100 images can essentially function as a dataset of 1000+ images, significantly improving the classification accuracy for cancer detection.

6b. Elaborate how pooling can be used to detect the edges in an image. Apply max and average pooling considering the size of 3×3 . (08 Marks)

*Clarification on the Question: The question "how pooling can be used to detect edges" is likely a phrasing error in the exam paper. Typically, **Convolution** with specific filters (like the one provided: $\begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$) is used to detect edges. **Pooling** is used for down-sampling and invariance. However, Max Pooling can highlight dominant features (like strong edges) in a local region.*

1. Role of Convolution/Pooling in Edge Detection: The filter provided in the problem implies a **Vertical Edge Detection** operation.

$$\text{Filter} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

When this filter is convolved with an image, it calculates the difference between the left and right pixels. Large differences result in high values, highlighting vertical edges. Max Pooling then helps by retaining only the strongest edge response in a neighborhood, making the edge detection robust to small shifts.

2. Application of Max and Average Pooling:

Given Input Matrix (6x6):

$$\begin{bmatrix} 3 & 0 & 1 & 2 & 7 & 4 \\ 1 & 5 & 8 & 9 & 3 & 1 \\ 2 & 7 & 2 & 5 & 1 & 3 \\ 0 & 1 & 3 & 1 & 7 & 8 \\ 4 & 2 & 1 & 6 & 2 & 8 \\ 2 & 4 & 5 & 2 & 3 & 9 \end{bmatrix}$$

Parameters:

- **Window Size:** 3x3
- **Stride:** Assumed to be 3 (Non-overlapping) to fit the 6x6 matrix perfectly into 4 blocks (2x2 output).

A. Max Pooling (3x3, Stride 3)

- **Block 1 (Top-Left):**

$$\begin{bmatrix} 3 & 0 & 1 \\ 1 & 5 & 8 \\ 2 & 7 & 2 \end{bmatrix}$$

Max = 8

- **Block 2 (Top-Right):**

$$\begin{bmatrix} 2 & 7 & 4 \\ 9 & 3 & 1 \\ 5 & 1 & 3 \end{bmatrix}$$

Max = 9

- **Block 3 (Bottom-Left):**

$$\begin{bmatrix} 0 & 1 & 3 \\ 4 & 2 & 1 \\ 2 & 4 & 5 \end{bmatrix}$$

Max = 5

- **Block 4 (Bottom-Right):**

$$\begin{bmatrix} 1 & 7 & 8 \\ 6 & 2 & 8 \\ 2 & 3 & 9 \end{bmatrix}$$

Max = 9

Max Pooling Output:

$$\begin{bmatrix} 8 & 9 \\ 5 & 9 \end{bmatrix}$$

B. Average Pooling (3x3, Stride 3)

- **Block 1 (Top-Left):** Sum = $3 + 0 + 1 + 1 + 5 + 8 + 2 + 7 + 2 = 29$ Average = $29/9 \approx 3.22$
- **Block 2 (Top-Right):** Sum = $2 + 7 + 4 + 9 + 3 + 1 + 5 + 1 + 3 = 35$ Average = $35/9 \approx 3.89$

- **Block 3 (Bottom-Left):** Sum = $0 + 1 + 3 + 4 + 2 + 1 + 2 + 4 + 5 = 22$ Average = $22/9 \approx 2.44$
- **Block 4 (Bottom-Right):** Sum = $1 + 7 + 8 + 6 + 2 + 8 + 2 + 3 + 9 = 46$ Average = $46/9 \approx 5.11$

Average Pooling Output:

Solutions: Artificial Neural Networks and Deep Learning

RV COLLEGE OF ENGINEERING® VI Semester B. E. Examinations Sept/Oct – 2024 Quiz:
PART-D

7a. Explain the concept of Back Propagation Through Time (BPTT) and its role in training RNNs. How does it address the issue of learning long term dependence? (06 Marks)

Concept of Back Propagation Through Time (BPTT): BPTT is the specific training algorithm used to update weights in Recurrent Neural Networks (RNNs). Because RNNs process sequential data, they cannot be treated as static feed-forward networks.

1. **Unfolding/Unrolling:** BPTT conceptually "unfolds" the RNN over time. If a sequence has T time steps, the RNN is visualized as a deep neural network with T layers, where each layer represents one time step.
2. **Weight Sharing:** Unlike a standard deep network where each layer has distinct weights, an unfolded RNN shares the *same* weight matrices (W_{input} , W_{hidden} , W_{output}) at every time step.
3. **Gradient Calculation:**
 - The network makes a forward pass through the entire sequence to calculate outputs and errors at each step.
 - The total error is the sum of errors at each time step: $E_{total} = \sum E_t$.
 - During the backward pass, gradients are propagated backward from the last time step ($t = T$) to the first ($t = 0$).
 - Because the weights are shared, the final gradient for a weight matrix is the sum of the gradients calculated at each time step.

Role in Training: BPTT is essential because it allows the network to understand how an input at time $t = 0$ influences the output at time $t = T$. It calculates the partial derivatives $\frac{\partial E}{\partial W}$ required for Gradient Descent optimization.

Addressing Long-Term Dependence: BPTT addresses the issue of learning temporal dependencies by establishing a **connected computational graph** from the present back to the past.

- By unrolling the network, a path is created where the error signal can flow from the current output back to previous inputs.
- This theoretically allows the network to learn that an event at $t = 1$ is the cause of an error at $t = 100$.
- *Note on Limitation:* While BPTT provides the **mechanism** to connect long-term dependencies, standard RNNs often struggle to learn them effectively due to the **Vanishing Gradient Problem** (where gradients become too small to update weights

effectively as they travel back many steps). This is why architectures like LSTMs are used in conjunction with BPTT.

7b. Articulate the role of RNNs in natural language processing (NLP) tasks. Identify and elaborate an appropriate RNN model for language translation with an example. (10 Marks)

Role of RNNs in NLP: Natural Language Processing deals with sequences of words or characters which have two key properties:

1. **Variable Length:** Sentences can be short ("Hi") or long ("The quick brown fox...").
2. **Context Dependence:** The meaning of a word depends on the words before (and sometimes after) it.

RNNs are ideally suited for NLP because:

- They process data sequentially, maintaining a "hidden state" (memory) that captures the context of the sentence seen so far.
- They can handle inputs of arbitrary lengths by iterating the same recurrence relation.
- They model the probability of the next word given the previous words $P(w_t | w_{t-1}, \dots, w_1)$, which is the foundation of Language Modeling.

Appropriate Model for Translation: Sequence-to-Sequence (Seq2Seq) / Encoder-Decoder

Language translation involves mapping an input sequence (Source Language) to an output sequence (Target Language) where the lengths may differ. The standard RNN model for this is the **Encoder-Decoder Architecture**.

Architecture Elaboration:

1. **Encoder RNN:**
 - Reads the input sentence one word at a time.
 - Updates its hidden state after every word.
 - The final hidden state (Context Vector) summarizes the semantic meaning of the entire input sentence.
2. **Context Vector:**
 - A fixed-length vector that acts as the "thought" or internal representation of the sentence.
3. **Decoder RNN:**
 - Initialized with the Context Vector.
 - Generates the output sentence one word at a time.
 - At each step, it uses its own previous output as the input for the next step.

Example: Translating English to French

- **Input:** "I am happy"
- **Process:**

1. **Encoder:**

- Input "I" → Update State h_1
- Input "am" → Update State h_2
- Input "happy" → Update State h_3 (This is the Context Vector).

2. **Decoder:**

- Input $\langle START \rangle$, State $h_3 \rightarrow$ Predict "Je"
- Input "Je", Current State $\rightarrow$ Predict "suis"
- Input "suis", Current State $\rightarrow$ Predict "heureux"
- Input "heureux" $\rightarrow$ Predict $\langle END \rangle$

- **Output:** "Je suis heureux"

8a. Analyze the differences between CNN and RNN (06 Marks)

Feature	CNN (Convolutional Neural Network)	RNN (Recurrent Neural Network)
Input Data Type	Grid-like Data: Spatial data such as Images ($H \times W \times C$) or Video frames.	Sequential Data: Temporal data such as Time Series, Audio, or Text/Sentences.
Connection Topology	Feed-Forward: Connections flow only from one layer to the next. No loops.	Recurrent: Connections form directed cycles (loops). Output of a node is fed back into itself.
Weight Sharing	Spatial Sharing: The same filter (kernel) slides over the entire image.	Temporal Sharing: The same weight matrices (W, U) are used at every time step.
Feature Extraction	Captures local spatial patterns (edges, textures, shapes). Features are translation invariant.	Captures temporal dependencies and sequential context (history).
Parallelization	High: The entire input image can be processed simultaneously (highly parallelizable on GPUs).	Low: Processing is inherently sequential; step t depends on step $t - 1$.
Primary Applications	Image Classification, Object Detection, Segmentation.	Machine Translation, Speech Recognition, Sentiment Analysis.

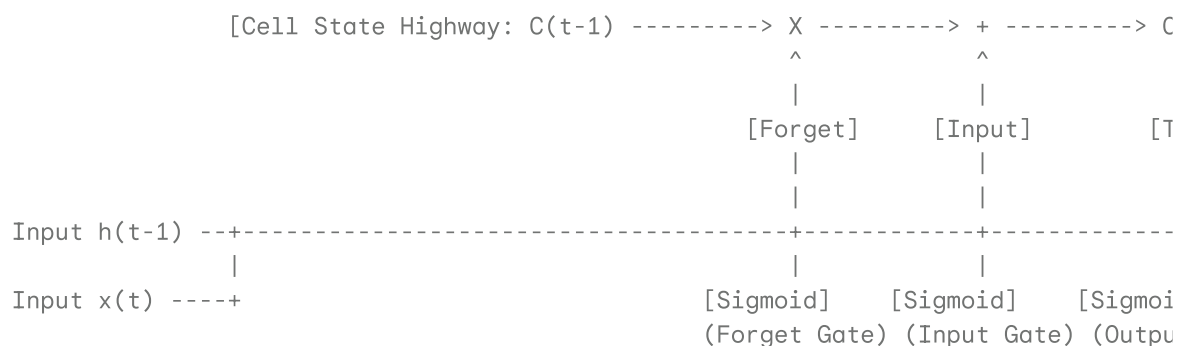
8b. With suitable example and necessary diagrams illustrate the working of LSTM (10 Marks)

Long Short-Term Memory (LSTM): Standard RNNs suffer from short-term memory due to vanishing gradients. LSTMs introduce a complex cell structure with **gates** to regulate the flow of information, allowing them to learn which data to keep or discard over long sequences.

Key Components (Gates): All gates use a Sigmoid activation (σ) to output values between 0 (block) and 1 (pass).

1. **Forget Gate (f_t):** "What should I forget from the past?"
 - Looks at previous hidden state (h_{t-1}) and current input (x_t).
 - Output 0 means "completely forget", 1 means "keep".
 - $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
2. **Input Gate (i_t) & Candidate Memory ($\tilde{C}_t$):** "What new info should I store?"
 - i_t decides which values to update.
 - $\tilde{C}_t$ creates a vector of new candidate values (using Tanh).
3. **Cell State Update (C_t):** The core memory "highway".
 - Multiply old state by forget gate ($f_t \times C_{t-1}$).
 - Add new candidate memory scaled by input gate ($i_t \times \tilde{C}_t$).
 - $C_t = f_t \times C_{t-1} + i_t \times \tilde{C}_t$
4. **Output Gate (o_t):** "What should I output to the next layer?"
 - Decides what parts of the cell state to output as the hidden state h_t .
 - $h_t = o_t \times \tanh(C_t)$

Diagrammatic Representation:



Illustrative Example: Task: Predicting the verb form in a sentence.

- **Context:** "The **boys**, who went to the park yesterday, **are** playing."
- **Step 1 (Input "The boys"):** The LSTM identifies "boys" is plural. The Input Gate stores this "plural subject" info in the Cell State (C_t).
- **Step 2 (Input "who went..."):** The network processes the dependent clause. The Forget Gate ensures the "plural subject" info is *retained* in the Cell State despite the intervening words.
- **Step 3 (Predicting "are"):** When the network needs to predict the verb, the Output Gate looks at the Cell State, sees the "plural subject" flag, and outputs "are" (plural) instead of "is".

Solutions: Artificial Neural Networks and Deep Learning

RV COLLEGE OF ENGINEERING® VI Semester B. E. Examinations Sept/Oct – 2024 Quiz:
PART-E

9a. Demonstrate the basic framework of Reinforcement learning (RL) and key components involved in the learning process. How do challenges such as exploration versus exploitation impact RL algorithms? (06 Marks)

Basic Framework of Reinforcement Learning: Reinforcement Learning is a type of machine learning where an **Agent** learns to make decisions by performing actions in an **Environment** and receiving feedback in the form of **Rewards** or penalties. Unlike supervised learning, there are no labeled input-output pairs; the agent learns through trial and error.

Key Components:

1. **Agent:** The learner or decision-maker (e.g., a robot, a chess program).
2. **Environment:** The external world the agent interacts with (e.g., the maze, the chessboard).
3. **State (S_t):** A representation of the current situation of the environment at time t .
4. **Action (A_t):** The move or decision made by the agent based on the current state.
5. **Reward (R_t):** A scalar feedback signal indicating how good or bad the action was. The goal is to maximize the cumulative reward.
6. **Policy (π):** The strategy or mapping from state to action that the agent employs.
7. **Value Function ($V(s)$):** An estimate of the total expected future reward starting from state s .

The Exploration vs. Exploitation Challenge: This is the fundamental dilemma in RL:

- **Exploration:** The agent tries **new, random actions** to discover potential rewards it hasn't seen before. This builds knowledge about the environment but risks receiving low rewards in the short term.
- **Exploitation:** The agent chooses the **best-known action** based on current knowledge to maximize immediate reward.

Impact on Algorithms:

- **Too much Exploration:** The agent never settles on an optimal policy and acts randomly, failing to maximize the score.
- **Too much Exploitation:** The agent gets stuck in a **local optimum**. It keeps repeating a "good enough" strategy without ever discovering a "great" strategy that might exist elsewhere.
- **Solution:** Algorithms often use ϵ -greedy strategies, where the agent explores with probability ϵ and exploits with probability $1 - \epsilon$, decaying ϵ over time.

9b. With an example, illustrate how a neural network can be used as a function approximator in an RL setting, such as playing Atari games. Describe how the neural network estimates the value function and benefits of this approach. (10 Marks)

The Problem with Traditional RL (Q-Tables): In simple games like Tic-Tac-Toe, we can use a **Q-Table** to store the value of every State-Action pair. However, in complex environments like Atari games (e.g., *Breakout* or *Pong*), the state is defined by screen pixels (210×160 pixels $\times$ 3 colors). The number of possible states is astronomically high ($256^{210 \times 160}$), making a table impossible to store or fill.

Neural Networks as Function Approximators (Deep Q-Networks - DQN): Instead of storing values in a table, we use a Deep Neural Network (CNN) to **approximate** the Q-value function:

$$Q(s, a; \theta) \approx Q^*(s, a)$$

where θ represents the weights of the neural network.

How it Works (Atari Example):

1. **Input:** The raw pixels of the game screen (often stacked 4 frames to capture motion) are fed into the network.
2. **Architecture:** Convolutional layers extract visual features (paddles, balls, walls). Fully connected layers process these features.
3. **Output:** The network outputs a vector of Q-values, one for each possible action (e.g., [Left: 0.2, Right: 15.5, Fire: 0.1]).
4. **Action Selection:** The agent picks the action with the highest output value (during exploitation).

Estimating the Value Function: The network is trained to minimize the difference (Loss) between its prediction and the "Target" Q-value derived from the Bellman Equation:

$$\text{Target} = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a'; \theta^-)$$

$$\text{Loss} = (\text{Target} - \text{Predicted } Q(S_t, A_t))^2$$

Backpropagation updates the weights θ to make the estimates more accurate over time.

Benefits of this Approach:

1. **Handling High-Dimensionality:** It can process raw sensory data (pixels) directly, eliminating the need for manual feature engineering.
2. **Generalization:** The network learns to generalize between similar states. If it learns that "ball moving toward paddle" is good in one position, it applies that knowledge to slightly different positions, which a table-based method cannot do.

10a. Consider the example "The player chases the ball". Apply Markov Decision Process and elaborate the steps in detail. (06 Marks)

To model "The player chases the ball" as an MDP, we must define the tuple (S, A, P, R, γ) . Let's assume a simplified Grid World environment.

1. States (S): The set of all possible configurations of the environment.

- $S = \{(x_p, y_p), (x_b, y_b)\}$
- Where (x_p, y_p) is the player's coordinate and (x_b, y_b) is the ball's coordinate.
- Example State: Player at (1,1), Ball at (4,4).

2. Actions (A): The set of choices available to the player.

- $A = \{\text{Move Up, Move Down, Move Left, Move Right, Stay}\}$

3. Transition Probability (P): The probability of moving to state s' given action a in state s .

- $P(s'|s, a)$:
 - *Deterministic*: If player chooses "Up", they move to $(x, y + 1)$ with probability 1.0.
 - *Stochastic*: If the ground is slippery, "Up" might result in $(x, y + 1)$ with 0.8 prob and staying at (x, y) with 0.2 prob.

4. Rewards (R): The immediate feedback received.

- If Player Position == Ball Position: $R = +10$ (Goal achieved).
- For every other step: $R = -1$ (Time penalty to encourage the shortest path).

5. Discount Factor (γ): A value between 0 and 1 (e.g., 0.9) that determines the importance of future rewards.

- This ensures the player prefers catching the ball *now* rather than infinitely later.

Process: The MDP seeks to find a Policy $\pi(s)$ that tells the player which direction to move in every grid cell to minimize the steps taken to reach the ball.

10b. Analyze how DL techniques are used in building conversational systems such as chatbots. Also list the benefits and limitations of using DL for natural language understanding and generation. (10 Marks)

DL Techniques in Conversational Systems: Modern chatbots (like ChatGPT or Google Assistant) rely heavily on Deep Learning, specifically **Sequence-to-Sequence (Seq2Seq)** models and **Transformers**.

1. Architecture:

- **Encoder:** Processes the user's input text (utterance). It converts the sequence of words into mathematical vectors (embeddings) and processes them to create a "Context Vector" that captures the semantic meaning.
- **Attention Mechanism:** Allows the model to focus on specific important words in the input sentence when generating a specific word in the response (e.g., focusing on "tomorrow" when deciding the tense of the verb).
- **Decoder:** Takes the context and generates the response probability distribution one word at a time.

2. Training:

- The models are trained on massive datasets of human conversations. They use **Self-Attention** (in Transformers) to understand the relationship between every word in a sentence, regardless of distance.

Benefits of DL in NLP:

1. **Contextual Understanding:** Unlike keyword-based bots, DL models understand context (e.g., "bank" of a river vs. "bank" for money).
2. **End-to-End Learning:** They map directly from input text to output text without needing complex manual rule sets or grammar trees.
3. **Generative Capability:** They can generate novel, fluent sentences that were not present in the training data, allowing for creative and flexible responses.
4. **Handling Long Dependencies:** Architectures like LSTMs and Transformers can remember details mentioned earlier in a long paragraph.

Limitations of DL in NLP:

1. **Hallucinations:** Generative models can confidently state incorrect facts because they are optimizing for probability/fluency, not truth.
2. **Data Hunger:** They require massive amounts of compute power and data to train effectively.
3. **Bias:** The models learn biases (gender, racial, cultural) present in the training data and can reproduce them in conversation.
4. **"Black Box" Nature:** It is difficult to interpret *why* a deep neural network generated a specific response, making debugging hard.